

AUREON9 Review Workflows Wiring

This package wires together:

- signed document upload flow
- member document persistence
- verification queue state transitions
- approve / reject / request-more-documents actions
- audit logging for every governance action
- role-aware enforcement

It is designed to sit on top of the existing:

- aureon9-react-starter
- aureon9-backend-foundation
- aureon9-admin-review-module

1. Prisma Schema Extensions

Add these enums and fields to the existing Prisma schema.

```
enum ReviewQueueStatus {
  PENDING
  UNDER_REVIEW
  REQUESTED_MORE_DOCUMENTS
  ESCALATED
  APPROVED
  REJECTED
  SUSPENDED
}

enum ReviewActionType {
  CREATE_CASE
  ASSIGN_REVIEWER
  MOVE_TO_UNDER_REVIEW
  REQUEST_MORE_DOCUMENTS
  ESCALATE
  APPROVE
  REJECT
  SUSPEND
  ADD_NOTE
}

enum DocumentReviewStatus {
  RECEIVED
  UNDER_REVIEW
  ACCEPTED
}
```

```

    REJECTED
    REPLACEMENT_REQUIRED
}

model VerificationRecord {
    id                String                @id @default(cuid())
    memberProfileId   String
    requestedLevel    VerificationLevel
    status            VerificationStatus    @default(PENDING)
    queueStatus       ReviewQueueStatus    @default(PENDING)
    notes             String?
    reviewedByUserId  String?
    assignedReviewerId String?
    submittedAt       DateTime              @default(now())
    reviewedAt        DateTime?
    memberProfile     MemberProfile         @relation(fields:
[memberProfileId], references: [id])
    actions           ReviewAction[]
}

model MemberDocument {
    id                String                @id @default(cuid())
    memberProfileId   String
    documentType      String
    verificationPurpose String?
    fileUrl           String
    storageKey        String?
    fileName          String
    mimeType           String?
    sizeBytes         Int?
    reviewStatus      DocumentReviewStatus @default(RECEIVED)
    uploadedAt        DateTime              @default(now())
    memberProfile     MemberProfile         @relation(fields:
[memberProfileId], references: [id])
}

model ReviewAction {
    id                String                @id @default(cuid())
    verificationRecordId String
    actionType        ReviewActionType
    actorUserId       String?
    notes             String?
    payloadJson       Json?
    createdAt         DateTime              @default(now())
    verificationRecord VerificationRecord    @relation(fields:
[verificationRecordId], references: [id])
}

```

2. Storage Strategy

Use an abstraction so the app can work with:

- Cloudflare R2
- AWS S3
- Supabase Storage
- local dev disk

Recommended interface:

```
// src/lib/storage.ts
export interface SignedUploadResult {
  uploadUrl: string;
  fileUrl: string;
  storageKey: string;
  expiresIn: number;
}

export async function createSignedUpload(input: {
  folder: string;
  fileName: string;
  contentType: string;
}): Promise<SignedUploadResult> {
  throw new Error("Implement with S3/R2 provider");
}
```

Folder convention:

```
/aureon9/member-documents/{memberProfileId}/{timestamp}-{fileName}
```

3. Zod Validation

```
// src/lib/review-validation.ts
import { z } from "zod";

export const createSignedUploadSchema = z.object({
  memberProfileId: z.string().cuid(),
  fileName: z.string().min(1),
  contentType: z.string().min(3),
  documentType: z.string().min(2),
  verificationPurpose: z.string().optional(),
});

export const finalizeUploadSchema = z.object({
  memberProfileId: z.string().cuid(),
```

```

    fileName: z.string().min(1),
    documentType: z.string().min(2),
    verificationPurpose: z.string().optional(),
    fileUrl: z.string().url(),
    storageKey: z.string().min(1),
    mimeType: z.string().optional(),
    sizeBytes: z.number().int().positive().optional(),
  });

export const reviewDecisionSchema = z.object({
  verificationRecordId: z.string().cuid(),
  notes: z.string().min(2).optional(),
});

export const assignReviewerSchema = z.object({
  verificationRecordId: z.string().cuid(),
  reviewerUserId: z.string().cuid(),
  notes: z.string().optional(),
});

export const requestMoreDocsSchema = z.object({
  verificationRecordId: z.string().cuid(),
  notes: z.string().min(5),
  requiredDocuments: z.array(z.string()).min(1),
});

```

4. Permission Rules

```

// src/lib/review-permissions.ts
import { UserRole } from "@prisma/client";

export function canReview(role?: UserRole) {
  return [UserRole.SUPER_ADMIN, UserRole.EXECUTIVE,
    UserRole.LEGAL_COMPLIANCE, UserRole.QUALIFICATIONS].includes(role as
    UserRole);
}

export function canAssignReviewer(role?: UserRole) {
  return [UserRole.SUPER_ADMIN, UserRole.EXECUTIVE,
    UserRole.LEGAL_COMPLIANCE].includes(role as UserRole);
}

export function canEscalate(role?: UserRole) {
  return [UserRole.SUPER_ADMIN, UserRole.LEGAL_COMPLIANCE,
    UserRole.QUALIFICATIONS].includes(role as UserRole);
}

export function canApprove(role?: UserRole) {

```

```

    return [UserRole.SUPER_ADMIN, UserRole.EXECUTIVE,
UserRole.LEGAL_COMPLIANCE].includes(role as UserRole);
}

export function canReject(role?: UserRole) {
    return [UserRole.SUPER_ADMIN, UserRole.EXECUTIVE,
UserRole.LEGAL_COMPLIANCE].includes(role as UserRole);
}

```

5. Signed Upload Request API

```

// src/app/api/documents/upload-url/route.ts
import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authConfig } from "@lib/auth";
import { createSignedUploadSchema } from "@lib/review-validation";
import { createSignedUpload } from "@lib/storage";
import { db } from "@lib/db";
import { writeAuditLog } from "@lib/audit";

export async function POST(req: NextRequest) {
    const session = await getServerSession(authConfig);
    if (!session?.user) return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });

    const body = await req.json();
    const parsed = createSignedUploadSchema.safeParse(body);
    if (!parsed.success) {
        return NextResponse.json({ error: parsed.error.flatten() }, { status:
400 });
    }

    const member = await db.memberProfile.findUnique({ where: { id:
parsed.data.memberProfileId } });
    if (!member) return NextResponse.json({ error: "Member profile not
found" }, { status: 404 });

    const signed = await createSignedUpload({
        folder: `aureon9/member-documents/${member.id}`,
        fileName: parsed.data.fileName,
        contentType: parsed.data.contentType,
    });

    await writeAuditLog({
        actorUserId: (session.user as any).id,
        entityType: "MemberDocument",
        entityId: member.id,
        action: "REQUEST_SIGNED_UPLOAD_URL",
    });
}

```

```

    payloadJson: parsed.data,
  });

  return NextResponse.json({ data: signed });
}

```

6. Finalize Upload API

```

// src/app/api/documents/finalize-upload/route.ts
import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authConfig } from "@/lib/auth";
import { finalizeUploadSchema } from "@/lib/review-validation";
import { db } from "@/lib/db";
import { writeAuditLog } from "@/lib/audit";

export async function POST(req: NextRequest) {
  const session = await getServerSession(authConfig);
  if (!session?.user) return NextResponse.json({ error: "Unauthorized" }, {
    status: 401 });

  const body = await req.json();
  const parsed = finalizeUploadSchema.safeParse(body);
  if (!parsed.success) {
    return NextResponse.json({ error: parsed.error.flatten() }, { status:
400 });
  }

  const document = await db.memberDocument.create({
    data: {
      memberProfileId: parsed.data.memberProfileId,
      documentType: parsed.data.documentType,
      verificationPurpose: parsed.data.verificationPurpose,
      fileUrl: parsed.data.fileUrl,
      storageKey: parsed.data.storageKey,
      fileName: parsed.data.fileName,
      mimeType: parsed.data.mimeType,
      sizeBytes: parsed.data.sizeBytes,
    },
  });

  await writeAuditLog({
    actorUserId: (session.user as any).id,
    entityType: "MemberDocument",
    entityId: document.id,
    action: "FINALIZE_DOCUMENT_UPLOAD",
    payloadJson: parsed.data,
  });
}

```

```
    return NextResponse.json({ data: document }, { status: 201 });
  }
```

7. Queue Retrieval API

```
// src/app/api/review-queue/route.ts
import { NextRequest, NextResponse } from "next/server";
import { getSession } from "next-auth";
import { authConfig } from "@lib/auth";
import { db } from "@lib/db";
import { canReview } from "@lib/review-permissions";

export async function GET(req: NextRequest) {
  const session = await getSession(authConfig);
  const role = (session?.user as any)?.role;
  if (!canReview(role)) return NextResponse.json({ error: "Forbidden" }, {
    status: 403 });

  const url = new URL(req.url);
  const status = url.searchParams.get("status") || undefined;
  const queue = await db.verificationRecord.findMany({
    where: status ? { queueStatus: status as any } : undefined,
    include: {
      memberProfile: {
        include: {
          user: true,
          participantClass: true,
          tier: true,
          documents: true,
        },
      },
    },
    actions: { orderBy: { createdAt: "desc" }, take: 20 },
    orderBy: { submittedAt: "desc" },
  });

  return NextResponse.json({ data: queue });
}
```

8. Assign Reviewer API

```
// src/app/api/review-queue/assign-reviewer/route.ts
import { NextRequest, NextResponse } from "next/server";
import { getSession } from "next-auth";
```

```

import { authConfig } from "@lib/auth";
import { db } from "@lib/db";
import { assignReviewerSchema } from "@lib/review-validation";
import { canAssignReviewer } from "@lib/review-permissions";
import { writeAuditLog } from "@lib/audit";

export async function POST(req: NextRequest) {
  const session = await getServerSession(authConfig);
  const role = (session?.user as any)?.role;
  if (!canAssignReviewer(role)) return NextResponse.json({ error:
    "Forbidden" }, { status: 403 });

  const parsed = assignReviewerSchema.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error:
    parsed.error.flatten() }, { status: 400 });

  const updated = await db.verificationRecord.update({
    where: { id: parsed.data.verificationRecordId },
    data: {
      assignedReviewerId: parsed.data.reviewerUserId,
      queueStatus: "UNDER_REVIEW",
      actions: {
        create: {
          actionType: "ASSIGN_REVIEWER",
          actorUserId: (session?.user as any)?.id,
          notes: parsed.data.notes,
          payloadJson: { reviewerUserId: parsed.data.reviewerUserId },
        },
      },
    },
  });

  await writeAuditLog({
    actorUserId: (session?.user as any)?.id,
    entityType: "VerificationRecord",
    entityId: updated.id,
    action: "ASSIGN_REVIEWER",
    payloadJson: parsed.data,
  });

  return NextResponse.json({ data: updated });
}

```

9. Request More Documents API

```

// src/app/api/review-queue/request-more-documents/route.ts
import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";

```



```

import { authConfig } from "@lib/auth";
import { db } from "@lib/db";
import { requestMoreDocsSchema } from "@lib/review-validation";
import { canReview } from "@lib/review-permissions";
import { writeAuditLog } from "@lib/audit";

export async function POST(req: NextRequest) {
  const session = await getServerSession(authConfig);
  const role = (session?.user as any)?.role;
  if (!canReview(role)) return NextResponse.json({ error: "Forbidden" }, {
    status: 403 });

  const parsed = requestMoreDocsSchema.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error:
    parsed.error.flatten() }, { status: 400 });

  const updated = await db.verificationRecord.update({
    where: { id: parsed.data.verificationRecordId },
    data: {
      queueStatus: "REQUESTED_MORE_DOCUMENTS",
      actions: {
        create: {
          actionType: "REQUEST_MORE_DOCUMENTS",
          actorUserId: (session?.user as any)?.id,
          notes: parsed.data.notes,
          payloadJson: { requiredDocuments: parsed.data.requiredDocuments },
        },
      },
    },
  });

  await writeAuditLog({
    actorUserId: (session?.user as any)?.id,
    entityType: "VerificationRecord",
    entityId: updated.id,
    action: "REQUEST_MORE_DOCUMENTS",
    payloadJson: parsed.data,
  });

  return NextResponse.json({ data: updated });
}

```

10. Escalate API

```

// src/app/api/review-queue/escalate/route.ts
import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authConfig } from "@lib/auth";

```

```

import { db } from "@lib/db";
import { reviewDecisionSchema } from "@lib/review-validation";
import { canEscalate } from "@lib/review-permissions";
import { writeAuditLog } from "@lib/audit";

export async function POST(req: NextRequest) {
  const session = await getServerSession(authConfig);
  const role = (session?.user as any)?.role;
  if (!canEscalate(role)) return NextResponse.json({ error: "Forbidden" }, {
    status: 403 });

  const parsed = reviewDecisionSchema.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error:
    parsed.error.flatten() }, { status: 400 });

  const updated = await db.verificationRecord.update({
    where: { id: parsed.data.verificationRecordId },
    data: {
      queueStatus: "ESCALATED",
      actions: {
        create: {
          actionType: "ESCALATE",
          actorUserId: (session?.user as any)?.id,
          notes: parsed.data.notes,
        },
      },
    },
  });

  await writeAuditLog({
    actorUserId: (session?.user as any)?.id,
    entityType: "VerificationRecord",
    entityId: updated.id,
    action: "ESCALATE_REVIEW",
    payloadJson: parsed.data,
  });

  return NextResponse.json({ data: updated });
}

```

11. Approve API

```

// src/app/api/review-queue/approve/route.ts
import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authConfig } from "@lib/auth";
import { db } from "@lib/db";
import { reviewDecisionSchema } from "@lib/review-validation";

```

```

import { canApprove } from "@lib/review-permissions";
import { writeAuditLog } from "@lib/audit";

export async function POST(req: NextRequest) {
  const session = await getSession(authConfig);
  const role = (session?.user as any)?.role;
  if (!canApprove(role)) return NextResponse.json({ error: "Forbidden" }, {
    status: 403 });

  const parsed = reviewDecisionSchema.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error:
    parsed.error.flatten() }, { status: 400 });

  const record = await db.verificationRecord.findUnique({ where: { id:
    parsed.data.verificationRecordId } });
  if (!record) return NextResponse.json({ error: "Not found" }, { status:
    404 });

  const updated = await db.$transaction(async (tx) => {
    const next = await tx.verificationRecord.update({
      where: { id: record.id },
      data: {
        status: "APPROVED",
        queueStatus: "APPROVED",
        reviewedByUserId: (session?.user as any)?.id,
        reviewedAt: new Date(),
        actions: {
          create: {
            actionType: "APPROVE",
            actorUserId: (session?.user as any)?.id,
            notes: parsed.data.notes,
          },
        },
      },
    });

    await tx.memberProfile.update({
      where: { id: record.memberProfileId },
      data: { verificationLevel: record.requestedLevel },
    });

    return next;
  });

  await writeAuditLog({
    actorUserId: (session?.user as any)?.id,
    entityType: "VerificationRecord",
    entityId: updated.id,
    action: "APPROVE_REVIEW_CASE",
    payloadJson: parsed.data,
  });
}

```

```
    return NextResponse.json({ data: updated });
}
```

12. Reject API

```
// src/app/api/review-queue/reject/route.ts
import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authConfig } from "@/lib/auth";
import { db } from "@/lib/db";
import { reviewDecisionSchema } from "@/lib/review-validation";
import { canReject } from "@/lib/review-permissions";
import { writeAuditLog } from "@/lib/audit";

export async function POST(req: NextRequest) {
  const session = await getServerSession(authConfig);
  const role = (session?.user as any)?.role;
  if (!canReject(role)) return NextResponse.json({ error: "Forbidden" }, {
    status: 403 });

  const parsed = reviewDecisionSchema.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error:
    parsed.error.flatten() }, { status: 400 });

  const updated = await db.verificationRecord.update({
    where: { id: parsed.data.verificationRecordId },
    data: {
      status: "REJECTED",
      queueStatus: "REJECTED",
      reviewedByUserId: (session?.user as any)?.id,
      reviewedAt: new Date(),
      actions: {
        create: {
          actionType: "REJECT",
          actorUserId: (session?.user as any)?.id,
          notes: parsed.data.notes,
        },
      },
    },
  });

  await writeAuditLog({
    actorUserId: (session?.user as any)?.id,
    entityType: "VerificationRecord",
    entityId: updated.id,
    action: "REJECT_REVIEW_CASE",
    payloadJson: parsed.data,
  });
}
```

```
});

return NextResponse.json({ data: updated });
}
```

13. Frontend Upload Wiring

```
// client flow example
async function uploadDocument(file: File, memberProfileId: string,
documentType: string, verificationPurpose?: string) {
  const uploadUrlRes = await fetch("/api/documents/upload-url", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      memberProfileId,
      fileName: file.name,
      contentType: file.type,
      documentType,
      verificationPurpose,
    }),
  });

  const uploadUrlJson = await uploadUrlRes.json();
  const { uploadUrl, fileUrl, storageKey } = uploadUrlJson.data;

  await fetch(uploadUrl, {
    method: "PUT",
    headers: { "Content-Type": file.type },
    body: file,
  });

  const finalizeRes = await fetch("/api/documents/finalize-upload", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      memberProfileId,
      fileName: file.name,
      documentType,
      verificationPurpose,
      fileUrl,
      storageKey,
      mimeType: file.type,
      sizeBytes: file.size,
    }),
  });
}
```

```
    return finalizeRes.json();
}
```

14. Frontend Review Actions Wiring

```
async function approveCase(verificationRecordId: string, notes?: string) {
    return fetch("/api/review-queue/approve", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ verificationRecordId, notes }),
    }).then((r) => r.json());
}

async function rejectCase(verificationRecordId: string, notes?: string) {
    return fetch("/api/review-queue/reject", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ verificationRecordId, notes }),
    }).then((r) => r.json());
}

async function requestMoreDocuments(verificationRecordId: string, notes:
string, requiredDocuments: string[]) {
    return fetch("/api/review-queue/request-more-documents", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ verificationRecordId, notes, requiredDocuments }),
    }).then((r) => r.json());
}

async function escalateCase(verificationRecordId: string, notes?: string) {
    return fetch("/api/review-queue/escalate", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ verificationRecordId, notes }),
    }).then((r) => r.json());
}
```

15. Required UI Behaviors

Review Queue Screen

- fetch `/api/review-queue`
- filter by queue status, participant class, risk, requested level
- show action history from `ReviewAction`
- optimistic refresh after every decision

Upload Screen

- drag/drop file
- show progress bar
- finalize upload
- append new document row without reload

Role-Based Screen Guards

- hide buttons client-side by role
 - enforce server-side permissions regardless of client UI
-

16. Suggested Risk Logic

High-risk cases should automatically move to `ESCALATED` if:

- requested level is `CAPITAL_VERIFIED` or `GOVERNANCE_APPROVED`
 - participant class is `CAPITAL_PARTICIPANT`, `STRATEGIC_PARTNER`, or `INSTITUTIONAL_PARTICIPANT`
 - one or more core documents are missing
-

17. Notification Hooks

Recommended hooks after actions:

- `REQUEST_MORE_DOCUMENTS` -> notify member by email/in-app
 - `ESCALATE` -> notify executive/legal queue
 - `APPROVE` -> notify member + unlock next level permissions
 - `REJECT` -> notify member + preserve audit trail
-

18. Immediate Build Order

1. Prisma migration for review workflow extensions
 2. Storage provider implementation for signed uploads
 3. Upload URL + finalize endpoints
 4. Queue retrieval endpoint
 5. Assign reviewer / request docs / escalate / approve / reject endpoints
 6. Connect admin UI buttons to live endpoints
 7. Add notification service
 8. Add automated high-risk escalation rules
-

19. Governance Alignment

This wiring preserves the ODIEBOARD operating model:

- Marketing acquires and positions participants
- Qualifications governs tier and certification movement
- Production / R&D implements workflow logic
- Customer tracks onboarding and missing items
- Legal / Compliance governs verification, approval, rejection, and escalation
- Executive layer handles strategic and capital-grade cases